REPORT · 2026年10月17日

H3D 进度页假死诊断与修复报告

解析线程静默死亡 / 深度图计数串造像 / 两遍稠密

文保数字化研发组
2026年10月17日

目 录

一、事件概述与结论 4

1.1 三条独立的缺陷 4

二、现场勘察：管线是好的，界面是坏的 5

2.1 先看磁盘，而不是先看界面 5

2.2 时间线还原 5

三、根因一：解析线程漏导入（致命回归） 6

3.1 缺失的那一行 6

3.2 为什么这个错误完全静默 6

3.3 修复：补导入 + 异常兜底 6

四、根因二：深度图计数跨造像污染 8

4.1 一个遍历所有目录的统计函数 8

4.2 残留目录让 UI 显示「已完成」 8

4.3 修复：按造像隔离 + 正确的分母 8

五、附带发现：patch_match_stereo 跑两遍 10

5.1 证据 10

5.2 为什么必须区分 10

5.3 修复：按遍数标注 + 相位计数 10

六、修复清单 12

6.1 代码改动 12

6.2 最终的界面表达 12

七、验证 13

7.1 自动化测试 13

7.2 打包版冒烟 13

7.3 实机运行复核 13

八、经验与后续 14

8.1 三条可复用的经验 14
8.2 把「静默」变成「可见」的三个层次 14
8.3 后续方向 14

一、事件概述与结论

2026-10-17 20:36，用户使用 H3D v1.1.0-r5 对 **30 张兵马俑照片**执行纯照片模式建模（造像编号 relic_001BMYyyyy）。运行到第 18 分钟时，界面出现如下状态：日志区只有 **3 行**内容，状态行固定显示「深度图 30/30」，进度条不再变化。用户据此判断为**生成页面卡住**。

本次排查的结论与以往几次**完全不同**：管线没有失败，COLMAP 一直在正常计算，最终产物完好。失效的是**界面与子进程之间的信息通道**——一条缺失的 import 语句让解析线程在第一行日志后静默退出，而界面又因为一个跨造像的统计口径，从上一轮残留数据里读出了一个看似「已完成」的假进度。两者叠加，造成了「18 分钟毫无动静」的观感。

> 一句话总结：**「卡住」的不是重建，是汇报。**子进程写下的 115 行日志证明深度图从 1/30 稳定推进，但这些信息在第一行之后就没有再进入 GUI。

1.1 三条独立的缺陷

编号	缺陷	后果	严重度
D-1	runner.py 调用 substep_of 但未导入	解析线程抛 NameError 后静默结束，界面从此不再更新	致命
D-2	_depth_progress 遍历所有造像取最大值	读到上一轮残留的 30 张深度图，显示假的 30/30	高
D-3	打包版子进程 stdout 按 cp936 编码	[H3D] 启动行中文在日志区显示为乱码方块	中

此外在核对过程中还发现一个**会误导进度判断**的既有行为：COLMAP 的 **patch_match_stereo 会跑两遍**（第一遍 photometric、第二遍 geom_consistency）。两遍各处理 30 张，日志序号都从 1 数到 30，界面上会出现「深度图 30/30 → 深度图 1/30」的**进度倒退**。这一点在本轮一并修正（见第五章）。

二、现场勘察：管线是好的，界面是坏的

2.1 先看磁盘，而不是先看界面

排查的第一步是把「用户看到的现象」与「磁盘上的事实」分开。读取测试目录下的日志文件后，事实非常清楚：

```
E:\兵马俑照片\batch_gui.log  (115 行 / 21:02 时的快照)

2026-10-17 20:36:40 [INFO] [h3d.runner.batch_processor] □ S1 开始：relic_001BMYyyyy
2026-10-17 20:36:48 [INFO] [h3d.sfm_solve] 照片预处理：30 张（0 张未缩放）
2026-10-17 20:36:49 [INFO] [h3d.sfm_solve] S1 清理旧数据库：database.db
2026-10-17 20:36:49 [INFO] [h3d.sfm_solve] S1 清理旧 sparse / dense 工作区
2026-10-17 20:36:49 [INFO] [h3d.sfm_solve] S1 COLMAP[1/3]: feature_extractor ...
2026-10-17 20:38:24 [INFO] [h3d.sfm_solve] S1 COLMAP[1/3] 完成
2026-10-17 20:43:25 [INFO] [h3d.sfm_solve] S1 COLMAP[2/3] 完成
2026-10-17 20:43:26 [INFO] [h3d.sfm_solve] S1 注册图像 #3
2026-10-17 20:43:35 [INFO] [h3d.sfm_solve] S1 注册图像 #23
...
2026-10-17 20:56:07 [INFO] [h3d.sfm_solve] S1 深度图 1/30：DSC_0008.JPG
2026-10-17 20:56:54 [INFO] [h3d.sfm_solve] S1 深度图 2/30：DSC_0009.JPG
...
2026-10-17 21:02:08 [INFO] [h3d.sfm_solve] S1 深度图 9/30：DSC_0016.JPG
```

日志显示 S1 走完了全部三个阶段，深度图以**每张约 46 秒**的节奏稳定推进。这是「管线正常」的铁证。

而 GUI 侧同一时刻只显示了三行：

```
□ 项目：rrrrr  根目录：E:\兵马俑照片
□ 配置已写入：E:\兵马俑照片\config.yaml
[H3D] □□□□□□□□□□ 26048 □□□L¼□□C:\Users\...\_MEI000068042
```

前两行是 GUI 自己在点击「开始」时写进日志区的，第三行是子进程输出的第一行——此后**一条都没有再进来**。

2.2 时间线还原

时刻	阶段	耗时	GUI 是否显示
20:36:39	子进程启动（PID 26048）	—	仅显示解包目录一行
20:36:49 → 20:38:24	特征提取 30 张	1.6 分钟	否
20:38:25 → 20:43:25	特征匹配（exhaustive）	5.0 分钟	否
20:43:26 → 20:56:07	稀疏重建（mapper）	12.7 分钟	否
20:56:07	去畸变	< 1 秒	否
20:56:07 → 21:17:09	深度图第一遍（30 张）	21.0 分钟	否
21:17:56 →	深度图第二遍（几何校验）	进行中	否

累计 41 分钟内，界面没有收到任何一条进度信息。这正是用户所说的「18 分钟没有任何反应」的真实来源。

> 判别方法：**先确认子进程是否还在算**（任务管理器看 colmap.exe 的 CPU、看 batch_gui.log 是否在增长），再看界面。二者不一致时，问题一定在通信层，而不是算法层。

三、根因一：解析线程漏导入（致命回归）

3.1 缺失的那一行

r5 为了让生成页显示子步骤（「稠密 2/3 深度图批量计算」这类文字），在 **src/h3d/gui/runner.py** 的日志循环里调用了解析函数 **substep_of()**，却忘了把它加进 import 列表：

```
from h3d.gui.logparse import (STAGE_ORDER, done_summary, stage_done_of,
                              stage_failed_of, stage_start_of)
#                             ↑少了 substep_of

...
for line in self._proc.stdout:
    line = line.rstrip()
    self.log_line.emit(line)        # 第一行：成功
    ...
    sub = substep_of(line)          # 第二行：NameError
```

3.2 为什么这个错误完全静默

这一行代码运行在 **QThread.run()** 里。Qt 的线程实现只捕获 C++ 层的异常，Python 层抛出的 **NameError** 会被 PySide6 打印到 stderr 后**终止该线程**，而打包版的 stderr 指向管道，进程被父进程视为正常输出流的一部分——GUI 主线程既收不到信号，也收不到异常。于是表现为：

环节	实际发生的事
日志区	不再追加任何新行（线程已死，无人转发）
进度条	停在启动时的 0%
阶段灯	S1 一直是「等待」灰色，不会变「运行中」橙色
状态行	停在初始文案「正在启动子进程…」或旧值
子进程	完全不受影响，继续算 40 分钟
退出后	GUI 既不报错也不完成，按钮一直是禁用态

> 这是本次最难排查的一类错误：**没有异常堆栈、没有失败日志、没有崩溃**，只有「什么都不发生」。排查时必须主动怀疑「线程是否还活着」，而不能只盯着子进程日志。

3.3 修复：补导入 + 异常兜底

除了补齐 import，更关键的是让**这类错误不再静默**。整个解析循环被包进 try/except，任何异常都会以醒目的错误行呈现在日志区，并触发失败汇总：

```
try:
    for line in self._proc.stdout:
        ...
except Exception as exc:
    # 关键：QThread.run 里未捕获的异常会让线程静默结束——GUI 侧
    # 只是"日志不再刷新、进度条不动"，看起来和卡死完全一样。
    import traceback
    self.log_line.emit(f"[错误] 日志解析线程异常：{exc}")
    self.log_line.emit(traceback.format_exc())
    self.finished_ok.emit(False, f"日志解析中断：{exc}")
    return
```

同时新增一条**结构性回归测试**：用 AST 解析 runner.py，把它从 logparse 导入的符号集合，与它实际需要的符号集合做差集。只要再有人加调用不加 import，测试立即失败。

```
def test_every_logparse_helper_used_is_imported(self):
    """回归锁：r5 漏导入 substep_of → QThread 首行即 NameError 静默死亡。"""
    tree = ast.parse(path.read_text(encoding="utf-8"))
    imported = set()
    for node in ast.walk(tree):
        if isinstance(node, ast.ImportFrom) and node.module == "h3d.gui.logparse":
            imported |= {a.asname or a.name for a in node.names}
    required = {"STAGE_ORDER", "stage_start_of", "stage_done_of",
                "stage_failed_of", "done_summary", "substep_of",
                "item_of", "s1_fraction"}
    assert not (required - imported)
```

四、根因二：深度图计数跨造像污染

4.1 一个遍历所有目录的统计函数

r5 为了让稠密阶段「有东西在动」，每 3 秒扫描一次深度图产物目录。原实现遍历 **99_runs/sfm_tmp 下的每一个造像**，取最大值：

```
def _depth_progress(self) -> tuple[int, int]:
    root = os.path.join(self.cfg.root_dir, "99_runs", "sfm_tmp")
    best = (0, 0)
    for item in os.listdir(root):              # ← 所有造像
        dm = os.path.join(root, item, "dense", "stereo", "depth_maps")
        ...
        cand = (geom if geom else photo, max(geom, photo))
        if cand[0] > best[0]:
            best = cand
    return best
```

4.2 残留目录让 UI 显示「已完成」

测试目录下同时存在两个造像的工作区：

目录	来源	深度图数量	备注
sfm_tmp/relic_666	上一轮（16:18）运行	30 张（photometric）	残留
sfm_tmp/relic_001BMYyyyy	本轮（20:36）运行	0 张（当时正值稀疏阶段）	当前

函数取最大值，于是返回 (30, 30)。界面显示「深度图 30/30」，而且因为 30 已经是上限，**它会永远停在那里。**这个数字完全没有参考价值：当前造像当时连一张深度图都还没开始算。

> 统计口径必须与**当前任务**绑定。任何「遍历同级目录后聚合」的实现，在支持批量、支持续跑的系统里，都等于把**历史数据**混进了**当前进度。**

4.3 修复：按造像隔离 + 正确的分母

造像编号本来就在日志里：□ **S1 开始：relic_001BMYyyyy。**新增 item_of() 把它解析出来，通过新信号传给界面，统计函数只认这一个目录：

```
#: 阶段开始行携带造像编号
_STAGE_START_ITEM = re.compile(r"□\s*(?:S1|M[1-6]|X1|X2)\s*开始[：:]\s*(\S+)")

def item_of(line: str) -> str | None:
    m = _STAGE_START_ITEM.search(line)
    return m.group(1) if m else None


def depth_progress(root_dir: str, item: str, phase: int = 1) -> tuple[int, int]:
    if not item:
        return (0, 0)                  # 还没开始，不显示
    base = os.path.join(root_dir, "99_runs", "sfm_tmp", item)
    ...
    ext = ".geometric.bin" if phase >= 2 else ".photometric.bin"
    done = sum(1 for f in names if f.endswith(ext))
    total = len([f for f in os.listdir(photos_small) if is_image_file(f)])
    return (done, total)
```

分母也一并修正：原来用 **max(geom, photo)** 当总数，在第一遍刚起步时会算出「12/0」这种无意义比值。现在改用 **photos_small 的照片数**，与 COLMAP 自己的 Processing view n / N 同源，永远是 30。

另外补了一条显示规则：**只有进入稠密阶段才显示深度图计数**（由子步骤文本判定），S1 结束时清空。否则 M1~M6 阶段状态行会一直挂着「深度图 30/30」。

五、附带发现：patch_match_stereo 跑两遍

5.1 证据

核对上一轮完整运行的 COLMAP 日志时发现，「Processing view n / 30」出现了 **60 次**，序号序列是 1..30 然后再次 1..30：

```
$ grep -c "Processing view" colmap_step12.log
60

$ grep -oE "Processing view [0-9]+ / [0-9]+" colmap_step12.log | sed -n '29,33p'
Processing view 29 / 30
Processing view 30 / 30
Processing view 1 / 30      ← 第二遍开始
Processing view 2 / 30
Processing view 3 / 30
```

原因是 **PatchMatchStereo.geom_consistency true**：第一遍做 photometric 匹配（产出 *.photometric.bin），第二遍做几何一致性校验（产出 *.geometric.bin）。两遍的耗时接近，合起来才是真正的稠密总时长。

5.2 为什么必须区分

不做处理	后果
日志文本	界面出现「深度图 30/30 → 深度图 1/30」，看起来像重头再来
文件计数	max(geom, photo) 在第二遍一直等于 30，进度条彻底不动
进度条	s1_fraction 映射到顶后回退，视觉上进度往回缩
用户判断	「是不是出错了 / 卡死了」——与本次反馈完全一致

5.3 修复：按遍数标注 + 相位计数

_ProgressLogger 增加了遍数状态机：当看到序号回到 1、而此前已经到达过总数时，判定进入下一遍，并改写标签：

```
m = _VIEW_PROG_RE.search(s)
if m:
    cur, total = int(m.group(1)), int(m.group(2))
    if cur == 1 and self._max_view >= total:
        self._pass += 1                 # 1 -> 2
    self._max_view = max(self._max_view, cur)
    tag = "深度图" if self._pass <= 1 else "几何校验"
    text = f"{tag} {cur}/{total}：{name}"
```

界面侧同步引入相位（phase）：第一遍数 *.photometric.bin，第二遍数 *.geometric.bin，切换时计数归零。进度条的比例区间也重新划分，保证两遍之间单调衔接：

子步骤	进度区间（占 S1 内部）	实测耗时
特征提取	5% → 18%	1.6 分钟
特征匹配	18% → 30%	5.0 分钟
稀疏重建	30% → 36%	12.7 分钟
去畸变	36% → 42%	< 1 秒

子步骤	进度区间（占 S1 内部）	实测耗时
深度图（第一遍）	42% → 65%	21 分钟
几何校验（第二遍）	65% → 90%	约 21 分钟
融合	92% → 99%	约 1 分钟

> S1 在总进度里只占 1/9（约 11%），但它通常消耗全流程 80% 的时间。如果只在 S1 完成时才推进进度条，用户看到的就是**「一小时不动，然后突然跳到 11%」**。把 S1 内部再细分成 7 段并连续映射，是让进度条「可信」的关键。

六、修复清单

6.1 代码改动

文件	改动	目的
gui/runner.py	补 substep_of 导入	消除 NameError（根因一）
gui/runner.py	解析循环包 try/except + 失败上报	同类错误不再静默
gui/runner.py	新增 item_changed 信号	把当前造像编号传给界面
gui/runner.py	新增 _bump() 单调进度	子步骤与阶段完成两套口径不打架
gui/runner.py	child_env 增加 PYTHONIOENCODING	缓解子进程编码（辅助）
gui/logparse.py	新增 item_of()	解析 □ S1 开始：<编号>
gui/logparse.py	新增 s1_fraction()	S1 内部 0~1 连续进度
gui/logparse.py	_CN_PROG 增加「几何校验」	第二遍可识别
gui/app.py	depth_progress(root, item, phase)	按造像 + 相位统计（根因二）
gui/app.py	状态行心跳「最后输出 Ns 前」	零输出期间证明进程存活
gui/app.py	进度条文字带当前阶段	37% · S1
gui/app.py	失败时进度条归零	避免「跑完却报错」的错觉
gui/app.py	日志着色优先级重排	加载进度行提到阶段切换之前
gui/launcher.py	新增 _force_utf8_stdio()	修中文乱码（根因三）
sfm_solve.py	_ProgressLogger 两遍状态机	第二遍标「几何校验」

6.2 最终的界面表达

修复后，运行中状态行的完整形态如下（每一段都对应一种可验证的事实）：

```
● S1 运行中 · 深度图 11/30：DSC_0018.JPG · 已运行 00:18:42 · 最后输出 24s 前
                ↑ 正在处理的文件名    ↑ 墙钟耗时    ↑ 心跳（>30s 才出现）

进度条：  37% · S1
```

四类信息各自解决一个问题：

信息	解决什么	失效时用户会怎么想
子步骤 + 文件名	现在到底在干什么	「是不是没开始？」
n/N 计数	还剩多少	「怎么一直不动？」
已运行 hh:mm:ss	投入了多少时间	「我是不是等了很久了？」
最后输出 Ns 前	进程是否还活着	「是不是卡死了？」

七、验证

7.1 自动化测试

本次新增 6 项测试，全量测试规模达到 **547 项，0 失败 0 错误**：

测试类	项数	锁住的行为
TestItemOf	2	从阶段开始行解析造像编号
TestS1Fraction	2	S1 内部进度比例单调、可解析
TestChildEnvEncoding	1	子进程环境注入 UTF-8 变量
TestDepthProgressScopedToItem	3	只统计当前造像；无编号返回 0
TestRunnerImportCompleteness	1	AST 差集：调用必须已导入
TestRunnerStreamsSignals	2	假 Popen 跑完整 run()，断言四类信号；解析异常必须上报
TestForceUtf8Stdio	2	流被真的重绑为 UTF-8；None 流不崩
TestTwoPassDepthMaps	4	第二遍标「几何校验」；跨两遍进度不回退；相位 2 数 geometric.bin

其中 **TestRunnerStreamsSignals** 用假的 Popen 对象直接驱动 BatchRunner.run()，检查 item / substep / stage / progress 四路信号是否全部到达——这比单独测试解析函数更接近真实失效场景：只要信号链路上任何一环断了，它就会失败。

7.2 打包版冒烟

新 exe 跑最小点云项目，M1~M6 全通（EXIT=0），启动行中文正常：

```
[H3D] 批处理进程 20764 解包目录：C:\Users\15308\...\_MEI00005b642
2026-10-17 21:15:00 [INFO] [h3d.runner.batch_processor] 任务发现完成：1 尊造像
2026-10-17 21:15:01 [INFO] [h3d.io.pointcloud_io] 读取点云 ...\a.ply：200 点
2026-10-17 21:15:01 [INFO] [h3d.runner.batch_processor]  点间距估计 = 0.100076 m
EXIT=0
```

对照：修复前同一命令输出为 **[H3D] □□□□□□□□□□ 20764 □□□L¼□□...**，确认乱码已消除。

7.3 实机运行复核

用户在 r5 上的这次运行被全程跟踪，用于验证「管线正常」的判断：

时刻	磁盘事实	说明
20:38:24	colmap_step1.log 17722 B	特征提取完成
20:43:25	colmap_step2.log 1068 B，stderr 空	匹配完成，EXIT=0
20:56:07	sparse/0 五个 bin 齐全	稀疏重建完成
21:02:08	photometric=9	深度图稳定推进
21:17:09	photometric=30	第一遍完成
21:23:39	geometric=8	第二遍（几何校验）推进中

八、经验与后续

8.1 三条可复用的经验

一、后台线程的异常必须自己兜住。QThread.run() 里的 Python 异常不会传播到主线程，也不会弹窗，只会让线程停在那里。凡是「线程 + 循环解析」的地方，外层必须有 try/except 并把异常转成用户可见的信息。

二、进度统计必须绑定当前任务。「遍历同级目录取最大值」在支持批量与续跑的系统里，本质上是在拿历史数据冒充当前进度。任何聚合都必须先确定「当前是谁」。

三、外部程序的一轮迭代未必是一次循环。COLMAP 的 patch_match_stereo 内部跑两遍，日志序号会回到 1。只要外部工具会多次打印「1/N」，界面就需要能识别遍数，否则进度倒退是必然而不是偶然。

8.2 把「静默」变成「可见」的三个层次

层次	手段	本次对应
结构层	AST 差集测试，静态发现调用与导入不一致	TestRunnerImportCompleteness
运行层	解析循环 try/except + 失败信号	日志解析线程异常
表达层	心跳（最后输出 Ns 前）+ S1 内部细分进度	状态行 / 进度条

8.3 后续方向

1. **真实文物批量实机校验**：8~20 张、重叠 70% 以上，重点看 S1 注册率、M5 纹理覆盖率（目标 60% 以上）。

2. **S1 单相机假设**：当前写死 --ImageReader.single_camera 1，多机位混拍时会导致相机模型误合并，需按 EXIF 分组或置 0。

3. **稠密阶段的耗时提示**：可在启动时根据照片数与历史数据估算总时长，在状态行显示「预计还需 18 分钟」，进一步降低焦虑。

4. **M4 纹理覆盖率复核**：示例数据仅 12.6%，需检查 UV 图集利用率与 Z-buffer 剔除阈值。

> 本次修复没有改动任何重建算法。所有改动集中在**信息的产生与呈现**上——这也是「进度页假死」这一类问题的通用解法：**先让事实可见，再谈优化。**